

You Place I Grab

Gameplay Introduction

You Place I Grab gameplay path: dofbot_ws/src/dofbot_color_sorting/你放我抓Grab play.ipynb

Start the code block, the robotic arm automatically moves to the recognition position. Place the block in the gray grid. After the robotic arm correctly recognizes 10 times continuously, it automatically grabs and places it in the corresponding color grid.

- Import header files

```
#!/usr/bin/env python
# coding: utf-8
import cv2 as cv
import threading
from time import sleep
from dofbot_config import *
import ipywidgets as widgets
from IPython.display import display
from color_sorting import color_sorting
```

- Initialize robotic arm position

```
import Arm_Lib
Arm = Arm_Lib.Arm_Device()
joints_0 = [90, 135, 0, 45, 90, 30]
Arm.Arm_serial_servo_write6_array(joints_0, 1000)
```

- Create instance, initialize parameters

```
# Create instance
sorting = color_sorting()
# Initialize mode
model = 'General'
# Color HSV threshold
color_hsv = {"red" : ((0, 43, 46), (10, 255, 255)),
             "green" : ((35, 43, 46), (77, 255, 255)),
             "blue" : ((100, 43, 46), (124, 255, 255)),
             "yellow": ((26, 43, 46), (34, 255, 255))}
# HSV parameter path
HSV_path="/home/yahboom/dofbot_ws/src/dofbot_color_sorting/HSV_config.txt"
# Read HSV configuration file, update HSV values
try: read_HSV(HSV_path,color_hsv)
except Exception: print("Read HSV_config Error!!!")
```

- Create controls

```
# Create control layout
button_layout = widgets.Layout(width='200px', height='70px',align_self='center')
# Output print
output = widgets.Output()
# Exit button
exit_button=widgets.Button(description='Exit',button_style='danger',layout=button_layout)
# Image control
imgbox = widgets.Image(format='jpg', height=480, width=640,
layout=widgets.Layout(align_self='center'))
# Vertical placement
controls_box = widgets.VBox([imgbox,
exit_button],layout=widgets.Layout(align_self='center'))
# ['auto','flex-start', 'flex-end', 'center', 'baseline', 'stretch', 'inherit',
'initial', 'unset']
```

- Control button

```
def exit_button_callback(value):
    global model
    model = 'Exit'
# with output: print(model)
exit_button.on_click(exit_button_callback)
```

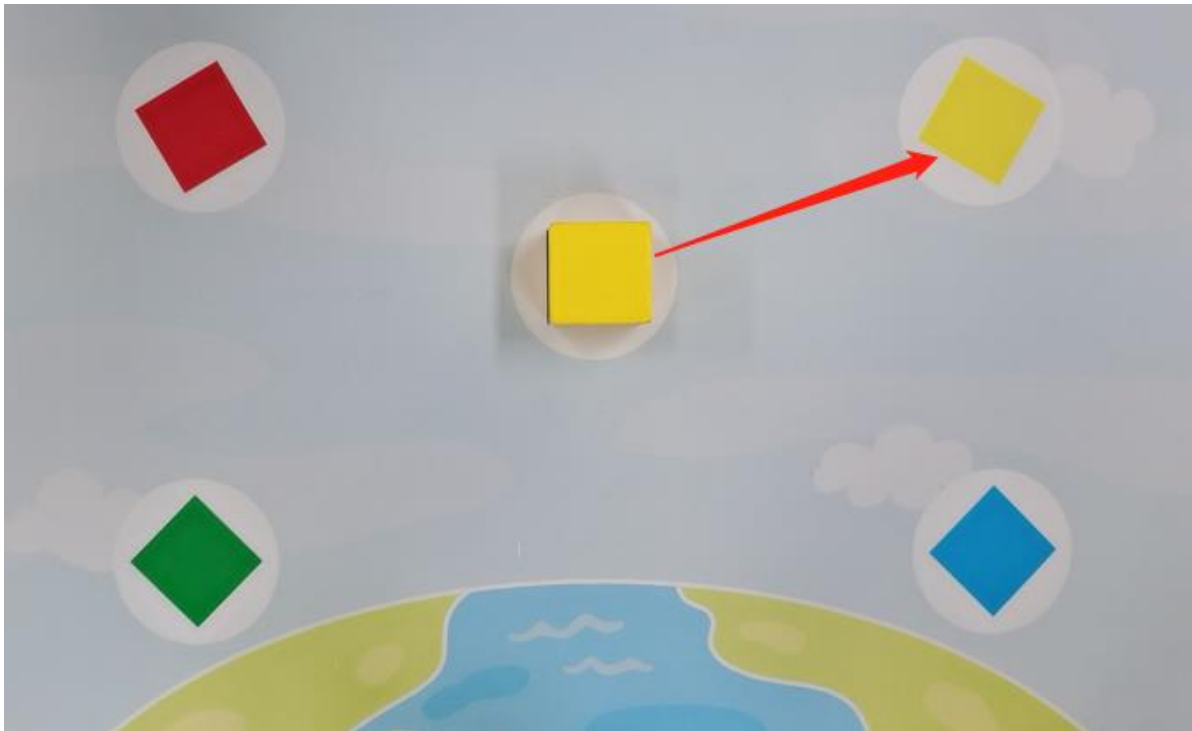
- Main program

```
def camera():
    # Open camera
    capture = cv.VideoCapture(0)
    # Loop while camera is opened normally
    while capture.isOpened():
        try:
            # Read each frame of the camera
            _, img = capture.read()
            # Uniform image size
            img = cv.resize(img, (640, 480))
            # Get motion information
            img = sorting.Sorting_grap(img, color_hsv)
            if model == 'Exit':
                cv.destroyAllWindows()
                capture.release()
                break
            # Add text
            imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
        except KeyboardInterrupt:capture.release()
```

- Start

```
display(controls_box,output)
threading.Thread(target=camera, ).start()
```

- Example image



For detailed gameplay code, see [dofbot_ws/src/dofbot_color_sorting/你放我抓.ipynb](#)

Library Code Design

- HSV filter target color

```
# Convert image to HSV
HSV_img = cv.cvtColor(mask, cv.COLOR_BGR2HSV)
# Filter elements between two arrays
img = cv.inRange(HSV_img, lowerb, upperb)
# Set non-mask detection part to black
mask[img == 0] = [0, 0, 0]
```

- Morphological transformation

```
# Get structuring elements of different shapes
kernel = cv.getStructuringElement(cv.MORPH_RECT, (5, 5))
# Morphological closing operation
dst_img = cv.morphologyEx(mask, cv.MORPH_CLOSE, kernel)
```

- Find contours in binary image

```
# Convert image to grayscale
dst_img = cv.cvtColor(dst_img, cv.COLOR_RGB2GRAY)
# Image binarization operation
ret, binary = cv.threshold(dst_img, 10, 255, cv.THRESH_BINARY)
# Get contour point set (coordinates)
contours, hierarchy = cv.findContours(binary, cv.RETR_EXTERNAL,
cv.CHAIN_APPROX_SIMPLE)
```

- Calculate contour bounding box

```
for i, cnt in enumerate(contours):  
    # BoundingRect function calculates bounding box values, x, y are coordinates,  
    w, h are width and height of rectangle  
    x, y, w, h = cv.boundingRect(cnt)  
    # Calculate contour area  
    area = cv.contourArea(cnt)
```

Obtain target, drive robotic arm to grab, grabbing process is as follows:

Lift robotic arm -> Release gripper -> Move to block position -> Grip -> Lift -> Move to target position -> Release gripper -> Reset robotic arm.